

MiNES Variance-Weighted PMF Fusion: Implementation Specification

Adaptive Nonequilibrium Sampling Notes

May 5, 2026

Contents

1 Task: Implement a clean MiNES-only workflow with variance-weighted PMF fusion	2
1.1 Deliverables	2
2 1. New Python file: scripts/mines_variance_fusion.py	2
2.1 Goal	2
3 Required command-line interface	2
4 Core data structures	3
4.1 EnsembleWindow	3
4.2 EQCluster	4
4.3 NEQSegment	4
4.4 PMFPatch	4
5 Simulation helpers	4
6 PMF reconstruction logic	5
6.1 EQ cluster patch	5
6.2 NEQ MTS patch	6
7 Global PMF fusion	6
8 Adaptive MiNES growth logic	7
9 Rescue phase	8
10 Important behavior	9
10.1 Do not hard-stitch PMFs	9
10.2 Use minimum variance only to define patch uncertainty	9
10.3 Keep local gauges separate from global gauge	9
10.4 Write enough intermediate files	9
11 2. Bash script: scripts/run_mines_variance_fusion.sh	10
12 3. Jupyter notebook: notebooks/mines_variance_fusion_visualization.ipynb	10
12.1 Plot 1: Global PMF	11
12.2 Plot 2: Global variance	11
12.3 Plot 3: Window centers	11
12.4 Plot 4: Patch coverage	11
12.5 Plot 5: Local patch PMFs	11
12.6 Plot 6: Error profile	11
13 Acceptance criteria	12

1 Task: Implement a clean MiNES-only workflow with variance-weighted PMF fusion

You are working in the current repository for adaptive nonequilibrium sampling. The existing `adaptive_methods.py` is too broad and does not implement the intended MiNES algorithm cleanly. Do **not** modify `adaptive_methods.py` unless absolutely necessary. Instead, create a new MiNES-only implementation.

1.1 Deliverables

Create three new files:

1. `scripts/mines_variance_fusion.py`
2. `scripts/run_mines_variance_fusion.sh`
3. `notebooks/mines_variance_fusion_visualization.ipynb`

The new Python file should only handle MiNES. It should not include AUS logic.

2 1. New Python file: `scripts/mines_variance_fusion.py`

2.1 Goal

Implement MiNES as a chain of EQ clusters and NEQ segments, where every local reconstruction produces:

```
PMF patch:
  pmf(x)
  variance(x)
  coverage_mask(x)
  unknown additive offset
```

The final PMF should be obtained by **global inverse-variance weighted fusion of all local PMF patches**, not by hard stitching.

The high-level workflow is:

1. Perform EQ simulations at endpoint ensembles L0 and R0.
2. Perform NEQ switching between L0 and R0.
3. Use the selected leaping method to decide child ensembles L1, R1, then perform EQ simulations.
4. For every neighboring pair of ensembles/clusters:
 - If JS divergence is below the threshold, merge them into an EQ cluster and reconstruct an EQ MBAR patch.
 - If JS divergence is above the threshold, construct an NEQ MTS patch between them.
5. Continue growing children until the left and right frontiers cross or meet.
6. After crossing, enter the rescue phase.
7. In rescue, target the grid bin with the largest global fused variance.
8. Add a rescue EQ window or NEQ bridge depending on where the maximum-variance bin lies.
9. Reconstruct all patches and refit the global PMF after every rescue round.

3 Required command-line interface

The script should be executable as:

```
python scripts/mines_variance_fusion.py \
  --system-root <path> \
  --bin <path-to-neq_sim> \
  --seed 123 \
  --label mines_variance_fusion \
  --total-budget-steps 200000 \
  --t-neq 5000 \
  --n-neq-traj 100
```

Arguments:

<code>--system-root</code>	required
<code>--bin</code>	required, path to compiled simulation binary
<code>--seed</code>	required int
<code>--label</code>	default: mines_variance_fusion
<code>--total-budget-steps</code>	default: 200000
<code>--t-neq</code>	default: 5000
<code>--n-neq-traj</code>	default: 100
<code>--n-eq-steps</code>	default: 10000
<code>--eq-save-every</code>	default: 10
<code>--tail-fraction</code>	default: 0.9
<code>--q-next</code>	default: 0.9
<code>--alpha</code>	default: 2.0
<code>--x-leap</code>	default: 1.5
<code>--k-min</code>	default: 1.0
<code>--k-max</code>	default: 50.0
<code>--k-rescue</code>	default: 10.0
<code>--js-threshold</code>	default: 0.8
<code>--bin-width</code>	default: 0.1
<code>--max-generations</code>	default: 10
<code>--max-rescue-rounds</code>	default: 5
<code>--n-bootstrap-eq</code>	default: 64
<code>--n-bootstrap-neq</code>	default: 64
<code>--variance-floor</code>	default: 1e-6

Output root:

```
<system-root>/MINES/<label>/raw/seed_<seed>/
```

4 Core data structures

Implement explicit data classes.

4.1 EnsembleWindow

```
@dataclass
class EnsembleWindow:
    name: str
    center_x: float
    k: float
    root: Path
    eq_file: Path
    tail_file: Path
    eq_rows: list[dict[str, str]]
    tail_rows: list[dict[str, str]]
```

```
x_most: float
generation: int
side: str # "left", "right", "rescue", "bridge"
```

4.2 EQCluster

```
@dataclass
class EQCluster:
    name: str
    windows: list[EnsembleWindow]
    left_x: float
    right_x: float
```

4.3 NEQSegment

```
@dataclass
class NEQSegment:
    name: str
    left: EQCluster | EnsembleWindow
    right: EQCluster | EnsembleWindow
    root: Path
    forward_trajectories: list[list[dict[str, str]]]
    reverse_trajectories: list[list[dict[str, str]]]
    forward_path_file: Path
    reverse_path_file: Path
```

4.4 PMFPatch

```
@dataclass
class PMFPatch:
    name: str
    kind: str # "EQ_MBAR" or "NEQ_MTS"
    grid: np.ndarray
    pmf: np.ndarray
    variance: np.ndarray
    coverage_mask: np.ndarray
    source_names: list[str]
    metadata: dict
```

The final global PMF should be constructed only from `PMFPatch` objects.

5 Simulation helpers

Reuse the existing simulator interface style from `adaptive_methods.py`.

Implement:

```
run_eq_window(...)
run_neq_protocol(...)
write_protocol_path(...)
read_csv_rows(...)
write_csv(...)
write_json(...)
load_json(...)
```

Use `run_context.json` from `system_root`.

The simulation binary should be called with the same low-level arguments used in the existing code:

```
-pot
-one-dimension
-thermal_kT
-dt
-gamma
-k0
-x0
-k1
-x1
-E1
-k
-center_xy
-eq_out
-T_eq
-eq_nout
-eq_seed
-out_dir
-log
```

For NEQ, use protocol path files and single-start trajectories, similar to the current implementation.

6 PMF reconstruction logic

6.1 EQ cluster patch

Implement:

```
build_eq_cluster_patch(cluster: EQCluster, grid: np.ndarray, ctx: dict, n_boot: int) -> PMFPatch
```

For a cluster with windows $m = 1, \dots, M$:

1. Reconstruct the base PMF by MBAR using all EQ tail samples from all windows in the cluster.
2. Bootstrap the EQ tail samples window-wise:
 - For each bootstrap replicate, resample each window's tail samples with replacement.
 - Reconstruct the MBAR PMF.
3. For each possible anchor window m :
 - Align every bootstrap PMF to zero at x_m^{most} .
 - Compute:

$$\text{var}_m(x) = \text{Var}_b[F_b(x) - F_b(x_m^{most})]$$

4. Define:

$$\text{var}_{EQ}(x) = \min_m \text{var}_m(x)$$

5. The base PMF can be stored in any consistent gauge, e.g. shifted to minimum zero. The global fusion step will determine patch offsets, so the local gauge does not need to be final.
6. The coverage mask should be finite PMF and finite variance.

Write patch files:

```
patches/<patch_name>_pmf.csv
patches/<patch_name>_bootstrap_variance.csv
patches/<patch_name>_summary.json
```

CSV columns:

```
x, pmf, variance, coverage, analytic_pmf
```

Also write per-anchor variance columns if useful:

```
var_anchor_<window_name>
```

6.2 NEQ MTS patch

Implement:

```
build_neq_mts_patch(segment: NEQSegment, grid: np.ndarray, ctx: dict, n_boot: int) -> PMFPatch
```

For each non-overlapping neighboring pair:

1. Run bidirectional NEQ.
2. Reconstruct PMF using bidirectional MTS.
3. Bootstrap the bidirectional MTS estimator.
4. Align each bootstrap PMF to the left anchor x_{left}^{most} and compute:

$$\text{var}_f(x)$$

5. Align each bootstrap PMF to the right anchor x_{right}^{most} and compute:

$$\text{var}_r(x)$$

6. Define:

$$\text{var}_{NEQ}(x) = \min(\text{var}_f(x), \text{var}_r(x))$$

7. Coverage mask should be finite PMF and finite variance.

Write patch files:

```
patches/<patch_name>_pmf.csv  
patches/<patch_name>_bootstrap_variance.csv  
patches/<patch_name>_summary.json
```

CSV columns:

```
x, pmf, variance, var_left_anchor, var_right_anchor, coverage, analytic_pmf
```

Use existing functions from analysis/bidirectional_mts_pmf.py where possible:

```
estimate_intermediate_reduced_free_energies  
build_bidirectional_mts_pmf  
bootstrap_bidirectional_mts_pmf  
align_pmf_to_reference  
align_pmf_to_endpoint_average_zero
```

Do not duplicate these algorithms unless necessary.

7 Global PMF fusion

Implement:

```
fit_global_pmf_from_patches(
    patches: list[PMFPatch],
    grid: np.ndarray,
    variance_floor: float,
    reference_x: float | None = None,
) -> tuple[np.ndarray, np.ndarray, dict]
```

The final PMF should solve:

$$\min_{G,c} \sum_p \sum_{x \in S_p} \frac{(G(x) - F_p(x) - c_p)^2}{\sigma_p^2(x) + \epsilon}$$

where:

- $G(x)$ is the global PMF.
- $F_p(x)$ is patch p 's PMF.
- c_p is the additive offset of patch p .
- $\sigma_p^2(x)$ is the patch variance.
- ϵ is `variance_floor`.

Fix the gauge by setting either:

```
G(reference_x) = 0
```

or, if `reference_x` is `None`:

```
min finite G(x) = 0
```

Also estimate approximate fused variance:

$$\text{var}_{\text{global}}(x) = \left[\sum_{p \in \mathcal{P}(x)} \frac{1}{\text{var}_p(x) + \epsilon} \right]^{-1}$$

This variance is approximate because patch errors are correlated, but it is useful for rescue targeting.

Write:

```
global_pmf.csv
```

with columns:

```
x, global_pmf, global_variance, global_std, n_covering_patches, analytic_pmf
```

Also write:

```
global_fit_summary.json
```

containing patch offsets, number of patches, RMS residuals, and gauge reference.

8 Adaptive MiNES growth logic

Implement a clean chain-based algorithm.

Initialize:

```
left_windows = [L0]
right_windows = [R0]
clusters = [cluster_L0, cluster_R0]
segments = []
patches = []
```

Step 1:

- EQ sample L0, R0.
- NEQ sample between L0, R0.
- Build an initial NEQ MTS patch.

Then for each generation:

1. Use the chosen leaping rule to propose **L_next** from the current left frontier and **R_next** from the current right frontier.
2. Run EQ for **L_next**, **R_next**.
3. Insert them into the chain.
4. Rebuild neighboring relationships.
5. For each neighboring pair:
 - Compute JS divergence from EQ tail samples.
 - If $JS < js_threshold$, merge into an EQ cluster and build one EQ MBAR patch.
 - Else, run NEQ if it does not already exist and build an NEQ MTS patch.
6. Fit global PMF from all patches.
7. Stop growth when the left and right frontiers cross or meet:
 - e.g. $L_next.center_x \geq R_next.center_x$, or
 - $L_next.x_most \geq R_next.x_most$, or
 - target-cross condition based on the chosen leaping rule.

Keep all decisions auditable. Write:

```
windows.csv
clusters.csv
segments.csv
patches.csv
generation_summary.csv
```

Each generation should have a directory:

```
generations/g00/
generations/g01/
...
```

9 Rescue phase

After the two ends meet/cross:

1. Fit the global PMF using all current patches.
2. Find:

$$x^* = \arg \max_x \text{var}_{global}(x)$$

among grid points with finite global PMF.

3. Add a rescue window near x^* :
 - center at nearest grid point to x^*
 - **k** = **k_rescue** by default

- optionally increase k if the target is not sampled in the tail quantile band.
4. Run EQ for the rescue window.
 5. Recompute clusters/segments/patches.
 6. Refit global PMF.
 7. Repeat until:
 - max rescue rounds reached,
 - total budget exhausted,
 - or maximum global variance falls below a practical threshold if implemented.

Write:

```
rescue_summary.csv
rescue_round_<n>/...
```

10 Important behavior

10.1 Do not hard-stitch PMFs

Do not implement final PMF as:

```
if var_f[x] < var_r[x]:
    use forward
else:
    use reverse
```

Do not implement final PMF as choosing one patch per bin.

Use variance-weighted global fusion.

10.2 Use minimum variance only to define patch uncertainty

Allowed:

```
var_EQ(x) = min_m var_m(x)
var_NEQ(x) = min(var_left_anchor(x), var_right_anchor(x))
```

Not allowed as the final PMF rule:

```
PMF(x) = PMF_from_lowest_variance_patch(x)
```

10.3 Keep local gauges separate from global gauge

Each patch can have arbitrary local gauge. The global fusion step must solve additive offsets.

10.4 Write enough intermediate files

The implementation should be easy to debug from disk outputs. For every generation, write:

```
window summaries
cluster summaries
segment summaries
patch PMF/variance files
global PMF file
```

11 2. Bash script: scripts/run_mines_variance_fusion.sh

Create a bash script that runs the new workflow with default parameters.

Usage:

```
bash scripts/run_mines_variance_fusion.sh
```

Optional arguments:

```
bash scripts/run_mines_variance_fusion.sh \  
  --system-root results/doublewell_1d \  
  --bin ./build/neq_sim \  
  --seed 10101 \  
  --label mines_variance_fusion_test
```

Defaults:

```
SYSTEM_ROOT="results/doublewell_1d"  
BIN="./build/neq_sim"  
SEED=10101  
LABEL="mines_variance_fusion"  
TOTAL_BUDGET_STEPS=200000  
T_NEQ=5000  
N_NEQ_TRAJ=100  
N_EQ_STEPS=10000
```

The script should:

1. Check that the Python file exists.
2. Check that `SYSTEM_ROOT/run_context.json` exists.
3. Check that the binary exists and is executable.
4. Create log directory:

```
<SYSTEM_ROOT>/MINES/<LABEL>/logs/
```

5. Run:

```
python scripts/mines_variance_fusion.py ...
```

6. Tee stdout/stderr to a timestamped log file.
-

12 3. Jupyter notebook: notebooks/mines_variance_fusion_visualization.ipynb

Create a notebook that visualizes the result.

It should allow the user to set:

```
system_root = "../results/doublewell_1d"  
label = "mines_variance_fusion"  
seed = 10101
```

The notebook should load:

```
<system_root>/MINES/<label>/raw/seed_<seed>/global_pmf.csv  
windows.csv  
clusters.csv
```

```
segments.csv
patches.csv
generation_summary.csv
rescue_summary.csv
```

Plots:

12.1 Plot 1: Global PMF

Plot:

```
x vs global_pmf
x vs analytic_pmf
uncertainty band using  $\pm \sqrt{\text{global\_variance}}$ 
```

12.2 Plot 2: Global variance

Plot:

```
x vs global_variance
```

Mark rescue target positions if available.

12.3 Plot 3: Window centers

Plot window centers over generation:

```
generation vs center_x
```

Color or marker by side:

```
left, right, rescue, bridge
```

12.4 Plot 4: Patch coverage

For each patch, show which grid points it covers.

A simple heatmap is fine:

```
x-axis: x
y-axis: patch name
value: coverage_mask
```

12.5 Plot 5: Local patch PMFs

Overlay all local patch PMFs after applying fitted offsets from `global_fit_summary.json`.

12.6 Plot 6: Error profile

If `analytic_pmf` is available:

```
x vs global_pmf - analytic_pmf
```

Also print RMSE over finite points.

The notebook should be robust if some files do not exist yet.

13 Acceptance criteria

The implementation is considered correct if:

1. It creates `scripts/mines_variance_fusion.py`.
2. It creates `scripts/run_mines_variance_fusion.sh`.
3. It creates `notebooks/mines_variance_fusion_visualization.ipynb`.
4. The Python script only implements MiNES, not AUS.
5. EQ-overlapping ensembles are merged into EQ clusters.
6. EQ clusters use MBAR PMF reconstruction.
7. EQ cluster variance is computed as:

$$\text{var}_{EQ}(x) = \min_m \text{var}_m(x)$$

where each $\text{var}_m(x)$ comes from bootstrap PMFs aligned to x_m^{most} .

8. Non-overlapping neighboring ensembles/clusters use bidirectional NEQ/MTS.
9. NEQ variance is computed as:

$$\text{var}_{NEQ}(x) = \min(\text{var}_{left-anchor}(x), \text{var}_{right-anchor}(x)).$$

10. The final PMF is not hard-stitched. It is obtained by inverse-variance weighted global fusion with fitted patch offsets.
11. The rescue target is selected from the maximum global fused variance.
12. All intermediate PMF, variance, window, cluster, segment, and rescue summaries are written to disk.
13. The bash script runs the workflow with sensible defaults.
14. The notebook visualizes global PMF, variance, windows, patches, rescue targets, and error against analytic PMF.

14 Notes

Prioritize correctness and clarity over cleverness. Keep the file modular. Avoid mixing old experimental protocols into this new implementation. This should become the clean reference implementation of MiNES with variance-weighted PMF fusion.